Prompt Engineering for Software Engineering Tasks
Zakariye Ali Mursal
PLP 2026

PROMPT ENGINEERING FOR SOFTWARE ENGINEERING TASKS

Student Name: Zakariye Ali Mursal
Registration Number: plp2026
Course: PLP
Unit Name: Software Engineering
Lecturer: Feisal
Institution: Garissa University
Date of Submission: 12/10/2026

EXERCISE A — EXPLAINING THE SDLC

Prompt 1: Zero-Shot Prompting

Prompt used:
"Explain the Software Development Life Cycle (SDLC) and describe its main phases."

AI Response:
1. Planning: Identifying the problem, project objectives, resources, costs, and schedule.
2. Requirements Analysis: Collecting and documenting user and system requirements.
3. System Design: Planning the software architecture, database, interface, and system components.
4. Development: Writing and integrating the program code.
5. Testing: Checking whether the software works correctly and identifying defects.
6. Implementation: Installing and releasing the software for users.
7. Maintenance: Correcting errors, improving performance, and updating the software after release.

The SDLC provides an organized approach to software development and helps teams manage project activities.

Summary:
The zero-shot prompt produced a clear but general explanation of the SDLC. It identified the seven phases in the correct order and briefly described each one. It did not provide a practical example, discuss team responsibilities, or mention risks. It would be useful for someone who already knows the SDLC but not for a beginner trying to learn it.

Prompt 2: Role Prompting

Prompt used:

"Act as a senior software engineer and university lecturer. Explain the seven phases of the Software Development Life Cycle (SDLC) in simple academic English. For each phase, explain its purpose, key activities, deliverables, and one possible risk. Use a practical example of developing a mobile application for university students."

AI Response:
The Software Development Life Cycle (SDLC) is a systematic process used to plan, build, test, release, and maintain software.

SDLC Phase	Purpose and Key Activities	Main Deliverables	Possible Risk
1. Planning	Define problem, objectives, budget, scope, schedule	Project proposal and initial plan	Unrealistic budget or schedule
2. Requirements Analysis	Interview users, document functional and non-functional requirements	Requirements specification	Important user needs may be missed
3. System Design	Design architecture, database, interfaces, components	Design documents and diagrams	Poor design may cause future problems
4. Development	Write, review, integrate code	Working software components	Coding errors and integration problems
5. Testing	Test functionality, security, performance, usability	Test cases and test reports	Serious defects may remain undetected
6. Implementation	Deploy the application and prepare users	Released application and documentation	Deployment failure or user resistance
7. Maintenance	Monitor, fix defects, introduce improvements	Updates, patches, maintenance records	Delayed fixes or security vulnerabilities

For example, when developing a mobile application for university students, the team would first identify student needs, design the application, build its features, test them, release the application, and maintain it after release.

Summary:
The role-prompted response was much more detailed and practical. It gave the purpose, activities, deliverables, and risks of every SDLC phase in a table, and it used a concrete example. The role of "senior software engineer and lecturer" helped the AI produce a structured, educational answer with clear formatting.

Comparison:
The role-prompted response was more useful. It covered the same phases but added deliverables, risks, and a practical example — content the zero-shot response skipped. Adding a role, a target audience, a format request, and a length limit turned a generic definition into a teachable explanation. The role set the tone; the format request enforced structure; the audience specification kept the language simple.

═══

EXERCISE B — SOFTWARE DEVELOPMENT MODELS

═══

Prompt used:

"You are helping a company choose a software development model for four different projects. For each situation below, recommend the most appropriate model, explain why it is appropriate, and give one limitation of that model:

1. A project with stable, clearly defined requirements.
2. A startup where requirements change frequently based on user feedback.
3. A large project with significant technical and security risks.
4. A project where testing and verification are extremely important.

Format your answer as four labelled sections: Situation 1, Situation 2, Situation 3, Situation 4."

AI Response:
Situation 1: Stable, Clearly Defined Requirements
Recommended model: Waterfall.
Why it is appropriate: Waterfall follows a sequential process where each phase is completed before the next begins. It suits projects where requirements are known and unlikely to change, since no rework is expected.
Limitation: Once development has started, changes to requirements are difficult and expensive to accommodate.

Situation 2: A Startup with Frequently Changing Requirements
Recommended model: Agile.
Why it is appropriate: Agile develops software in short iterations and incorporates user feedback continuously. This allows the team to adjust features as customer needs evolve.
Limitation: Because scope is flexible, predicting final cost and delivery date is difficult.

Situation 3: A Large Project with Significant Technical and Security Risks
Recommended model: Spiral.
Why it is appropriate: The Spiral model combines iterative development with risk analysis at each cycle. This helps identify and address technical and security risks early, before they grow.
Limitation: Spiral is expensive and requires skilled risk-management personnel; it can be overkill for smaller projects.

Situation 4: A Project Where Testing and Verification Are Extremely Important
Recommended model: V-Model.
Why it is appropriate: The V-Model pairs each development phase with a corresponding test phase, so verification and validation are planned from the start. This makes it well-suited to safety-critical or regulated systems.
Limitation: The V-Model is inflexible; changes to approved requirements after development has begun are costly.

Reflection:
The AI matched each situation to the model most commonly recommended for it. Waterfall fits stable requirements, Agile fits frequent change, Spiral fits high risk, and the V-Model fits testing-heavy projects. Because I asked for one limitation of each model, the response was balanced rather than promotional. This exercise reinforced that no single SDLC model is

universally correct — the right choice depends on how stable the requirements are, how much risk is involved, and how much emphasis the project places on testing. It also showed how a well-structured prompt produces a more useful comparison than a generic question would.

EXERCISE C — REQUIREMENTS EXTRACTION

Client brief (as given in the lab):
A local bookstore needs software to:
- Track books, quantities, and prices.
- Manage supplier payments.
- Alert staff when stock is low.
- Identify best-selling books.
- Allow customers to check availability online.
- Produce clear records for a part-time assistant.

The owner has a limited budget.

Prompt used:
"You are an experienced software requirements analyst.

Read the following client brief and extract the software requirements.

Client Brief:
A local bookstore needs software to:
- Track books, quantities, and prices.
- Manage supplier payments.
- Alert staff when stock is low.
- Identify best-selling books.
- Allow customers to check availability online.
- Produce clear records for a part-time assistant.

The owner has a limited budget.

Produce the following, in clearly labelled sections:
1. Functional requirements.
2. Non-functional requirements.
3. Must Have requirements (MoSCoW).
4. Should Have requirements.
5. Nice to Have requirements.
6. Questions or ambiguities that need clarification.

Use simple English. Number each requirement. Where the brief does not specify something, mark it as an assumption or a question rather than inventing a fact."

AI Response:

A. Functional Requirements

FR1. The system shall record books, including title, author, price, and quantity in stock.
FR2. The system shall allow staff to update book details and prices.
FR3. The system shall record supplier payments against the correct supplier and date.
FR4. The system shall alert staff when a book's stock falls below a defined threshold.
FR5. The system shall identify best-selling books based on recorded sales.
FR6. The system shall allow customers to check book availability online.
FR7. The system shall produce clear reports for the part-time assistant (e.g. daily sales, stock levels).

B. Non-Functional Requirements
NFR1. Usability: The system should be simple enough for the owner and part-time assistant to learn with minimal training.
NFR2. Affordability: The solution must fit the bookstore's limited budget — favour low-cost tools and hosting.
NFR3. Reliability: Stock and payment records must be stored and retrieved accurately.
NFR4. Security: Access to supplier payments must be restricted to authorised users.
NFR5. Performance: Availability lookups and report generation should respond within a few seconds.
NFR6. Maintainability: The system should be easy for a small team to update and support.

C. Must Have Requirements (MoSCoW)
- Record books, quantities, and prices.
- Record and manage supplier payments.
- Alert staff when stock is low.
- Track sales sufficiently to identify best-selling books.
- Allow customers to check book availability online.
- Produce clear bookstore records.

D. Should Have Requirements
- Search for books by title or author.
- Generate simple stock and sales reports.
- Restrict access to supplier payment records.

E. Nice to Have Requirements
- Automatic email or SMS alerts when stock is low.
- Simple charts showing sales trends.
- Online payment integration for customers.

F. Questions and Ambiguities
Q1. What information must be stored for each book (title, author, ISBN, publisher, edition)?
Q2. How is the low-stock threshold set — a single value for all books, or per book?
Q3. Should the system record only payments made, or also invoices and outstanding balances?
Q4. How will sales be recorded so that "best-seller" can be calculated?
Q5. Should customers see exact stock quantities, or only "In stock" / "Out of stock"?
Q6. What specific records does the part-time assistant need to produce (daily, weekly, monthly)?
Q7. What is the actual budget range for development, hosting, and ongoing support?

Q8. Should customers be able to buy online, or only check availability?

Evaluation:
Two things the AI did well: It separated functional from non-functional requirements and applied MoSCoW prioritisation exactly as requested. It also flagged ambiguities — such as whether customers see exact stock quantities — instead of inventing a policy.

One thing the AI missed or miscategorised: The Should Have and Nice to Have lists contain features (like online payment integration and SMS alerts) that the brief never mentioned. These are reasonable suggestions, but they need the owner's approval before being treated as requirements.

One improvement I would make to my prompt: I would add — "Do not treat suggested features as confirmed requirements. Clearly separate directly stated requirements from recommendations, and list any suggested feature under questions rather than under Must Have or Should Have."

═══

EXERCISE D — PROMPT ITERATION FOR A UNIVERSITY STUDY-ROOM BOOKING MOBILE APP

═══

Introduction:
Prompt iteration is the process of improving a prompt through repeated testing and refinement. This exercise demonstrates four versions of a prompt for developing a mobile application that allows university students to book campus study rooms.

VERSION 1 — BASIC PROMPT

Prompt used:
"Explain how to develop a mobile app that allows university students to book campus study rooms."

AI Response:
To develop a mobile study-room booking application, the development team should follow these steps: (1) identify the problem and objectives, (2) collect requirements from students and staff, (3) design the mobile UI, (4) develop the app and connect it to a database, (5) test it, (6) deploy it, and (7) maintain it. The app could allow students to view rooms, check availability, book, and cancel.

Weaknesses:
The response was too general. It listed SDLC steps but did not define features, supported platforms, booking rules, identity verification, team roles, duration, or budget.

VERSION 2 — IMPROVED PROMPT WITH REQUIREMENTS

What I changed and why:

Version 1 was too general. I added a role (business analyst + mobile developer), specific features, and requested functional requirements, non-functional requirements, user roles, and open questions.

Prompt used:
"Act as a business analyst and mobile application developer. I want to develop a mobile application for university students to book campus study rooms. Students should be able to search for rooms, view available time slots, make bookings, view their bookings, and cancel their own bookings. Identify the functional requirements, non-functional requirements, user roles, and important questions that must be answered before development. Present the answer in simple English and use tables where appropriate."

AI Response:
A. Functional Requirements: FR1 student login; FR2 room search; FR3 view availability; FR4 create booking; FR5 view bookings; FR6 cancel booking; FR7 prevent double-booking (no two confirmed bookings for the same room and overlapping slot); FR8 manage rooms (authorised staff); FR9 verify student identity before booking.

B. Non-Functional Requirements: security, performance, reliability, usability, compatibility (iOS + Android), availability during booking hours, maintainability, privacy.

C. User Roles: Student, University Administrator, System Administrator, Support Staff.

D. Questions to Clarify: which rooms; how long a booking can be; how far ahead bookings open; whether a student can hold multiple bookings; cancellation policy; which identity system; who manages room schedules; what happens if a room becomes unavailable; whether to send notifications; expected user scale.

Weaknesses remaining after Version 2:
Clearer requirements, but no development plan. Still missing SDLC phases, timeline, deliverables, risks, and team responsibilities.

VERSION 3 — PROMPT WITH A COMPLETE SDLC PLAN

What I changed and why:
Version 2 was clear on requirements but had no plan. I asked for a phase-by-phase SDLC plan with duration, deliverables, risks, mitigation, and responsibilities.

Prompt used:
"Act as a senior software project manager. Prepare a project plan for developing a mobile application that allows university students to search for campus study rooms, view schedules, make bookings, view their own bookings, and cancel permitted bookings. Organize the plan using the seven SDLC phases in order. For each phase, provide the estimated duration in weeks, main activities, key deliverables, one major risk, a mitigation measure, and the responsible team roles. Use simple English and present the result in a table. State any assumptions clearly."

AI Response:

SDLC Phase	Duration	Main Activities	Key Deliverables	Risk & Mitigation	Responsible Roles
1. Planning	1 week	Define objectives, scope, resources, schedule	Project charter, initial plan	Unclear scope → agree on features first	Project manager, university rep
2. Requirements Analysis	2 weeks	Gather needs, document booking rules	Requirements specification	Missing requirements → consult students, staff, IT	Business analyst, project manager
3. System Design	2 weeks	Design screens, architecture, database, APIs	Design docs, DB design, prototypes	Poor booking design → design review	UI/UX designer, backend dev, analyst
4. Development	5 weeks	Build app, backend, database, booking features	Working app, backend APIs	Integration problems → code review, integration tests	Mobile devs, backend dev, DB dev
5. Testing	2 weeks	Test features, usability, security, reliability	Test cases, defect reports, summary	Undetected defects → test main workflows	Tester, devs, security support
6. Implementation	1 week	Configure production, release	Released app, deployment docs	Release failure → deployment checklist + rollback	Deployment engineer, devs, PM
7. Maintenance	1 week + ongoing	Monitor, fix, improve	Maintenance report, issue log	Unreported issues → support process	Devs, IT support, PM

Estimated initial duration: 14 weeks.

Weaknesses remaining after Version 3:
Still missing: a rough budget, a detailed double-booking prevention strategy, and a student identity-verification approach.

VERSION 4 — FINAL REFINED PROMPT

What I changed and why:
Version 3 was a complete high-level plan but lacked a budget and technical controls. I added explicit constraints on double-booking, campus identity verification, iOS/Android support, a budget, and a request to label assumptions instead of inventing details.

Final prompt used:
"Act as a senior software project manager, business analyst, mobile application architect, and software quality assurance specialist. Prepare a professional project plan for a university study-room booking mobile application.

Project requirements:
1. The application must support both iOS and Android.
2. Students must be able to search for available campus study rooms.
3. Students must be able to view room schedules and book specific time slots.
4. Students must be able to view and cancel their own bookings according to university rules.
5. The system must prevent double-booking. If two students request the same room and overlapping time slot simultaneously, only one request may be confirmed.

6. The system must verify that a user is a current university student through the university's campus identity system before allowing a booking.
7. Authorized university staff must be able to manage rooms, schedules, and booking rules.

Your task:
- Organize the plan using all seven SDLC phases in the correct order.
- Provide an estimated duration in weeks for every phase.
- Identify the main activities and key deliverables for each phase.
- Identify project risks, especially simultaneous booking conflicts and identity-verification failures.
- Provide practical mitigation measures for each risk.
- Identify responsible team roles.
- Provide a rough budget breakdown in Kenyan shillings (KES).
- Present one row per SDLC phase in a clear table.
- Use simple academic English and label all assumptions clearly.
- Explain how the backend and database should prevent two students from confirming the same room and overlapping time slot.
- Explain how identity verification should work and what should happen if the campus identity service is unavailable.
- Include testing, deployment, and ongoing maintenance considerations.
- Do not invent university-specific identity system details. Identify them as items requiring confirmation with university IT."

AI Response:

A. Project Overview
A mobile application that enables university students to find and reserve campus study rooms. It supports iOS and Android, uses a backend to manage rooms, schedules, verification, and bookings, and integrates with the university's identity service.

Assumptions:
1. The university will provide access to a campus identity verification service.
2. The university will provide room information, schedules, and booking rules.
3. iOS/Android version targets will be confirmed during requirements analysis.
4. The application uses an online backend to validate bookings.
5. Schedule and budget figures are academic estimates, not a quotation.

B. Final SDLC Implementation Plan
SDLC Phase	Duration	Main Activities	Key Deliverables	Risks & Mitigation	Responsible Team
1. Planning	1 week	Define objectives, scope, resources, schedule	Project charter, risk register	Unclear objectives → agree scope with stakeholders	Project manager, university rep
2. Requirements Analysis	2 weeks	Interview students and staff, define booking rules	Requirements specification, user stories	Missing booking rules → consult IT and stakeholders early	Business analyst, PM, university IT

3. System Design	2 weeks	Design screens, backend, DB, APIs, security, conflict handling	Architecture, DB schema, prototypes, API spec	Design allows conflicting bookings → review DB constraints and transaction design	Designer, backend dev, analyst
4. Development	5 weeks	Build iOS/Android app, backend, DB, booking logic, identity integration	Working app, backend services, source code	Simultaneous booking or failed identity checks → server-side validation, DB transactions	Mobile devs, backend dev
5. Testing	2 weeks	Test functional, concurrency, identity, security, cancellations, usability, compatibility	Test plan, defect reports, security results	Booking conflicts or undetected defects → concurrent-request testing, regression tests	Tester, devs, university IT
6. Implementation	1 week	Configure production, connect identity service, publish, train staff	Released app, deployment checklist, rollback plan	Deployment or identity failure → rehearsal, rollback, monitoring	Deployment engineer, devs, PM
7. Maintenance	1 week + ongoing	Monitor, respond to incidents, fix defects, plan improvements	Maintenance report, issue log	Post-release failures → monitor, prioritise defects, support channel	Backend/mobile devs, IT support, PM

Total estimated initial duration: 14 weeks (including one week of post-release support). Ongoing maintenance continues after.

C. Preventing Double-Booking
1. A student selects a room and time slot.
2. The app sends a booking request to the backend.
3. The backend verifies identity and booking permissions.
4. The backend checks the room schedule and booking rules.
5. The database performs the booking inside a safe transaction (or equivalent concurrency control).
6. The booking is confirmed only if the slot is still available.
7. If another student has secured the slot, the second request is rejected with a clear message.

For predefined time slots, the database can enforce a unique constraint on room + slot for confirmed bookings. For arbitrary ranges, a transaction-and-overlap strategy is required. The app must never display a booking as confirmed before the backend has committed it.

D. Student Identity Verification
1. Student opens the app and chooses to sign in.
2. The app redirects to the university's approved identity service (or an approved secure integration).
3. The identity service verifies credentials and current student status.
4. The backend validates the returned authentication result and required permissions.
5. The student receives access to booking functions only after successful verification.

If verification fails:
Situation	Expected Response
Invalid credentials	Deny access, display clear error
Not a current student	Block booking
Identity service unavailable	Temporary message + retry option

| Session expired | Ask to authenticate again |
| Invalid identity response | Reject securely, log event |

If the identity service is unavailable, the app must not treat the student as verified. Fail securely and allow retry.

E. Team Roles and Responsibilities
Role	Responsibility
Project Manager	Scope, schedule, budget, risks, reporting
Business Analyst	Requirements, booking rules, acceptance criteria
UI/UX Designer	Mobile screens and booking workflows
Mobile Developers	iOS and Android interfaces, API integration
Backend Developer	Auth checks, booking logic, APIs, business rules
Database Developer	Schemas, indexes, constraints, transactions, backups
Software Tester	Functional, security, compatibility, concurrent-booking testing
University IT / Integration	Campus identity service and room data access
Deployment & Support	Production setup, release, monitoring, user support

F. Preliminary Budget Estimate (KES)
Item	Estimated Cost
Planning and Requirements Analysis	50,000
UI/UX and System Design	70,000
Mobile Application Development	200,000
Backend and Database Development	150,000
Campus Identity Integration	80,000
Testing and Security Checks	70,000
Deployment and Release	30,000
Initial Maintenance and Support	25,000
Hosting, Tools, Related Services	25,000
Contingency Reserve	80,000
Total Estimated Budget	**780,000**

Illustrative estimate only — not a supplier quotation.

G. Additional Risks
Risk	Effect	Mitigation
Double-booking	Two students hold the same slot	Server-side transactions, DB constraints, concurrency tests
Identity service failure	Verified students cannot book	Clear errors, controlled retries, monitoring, IT coordination
Unauthorized access	Users access bookings they don't own	Server-side authorization on every protected operation
Incorrect room schedules	Students book unavailable rooms	Confirm schedule ownership, authorised updates
Data loss	Booking data lost or corrupted	Backups, recovery procedures, monitoring

Schedule delays	Missed release date	Milestone tracking, prioritise essential features
Budget overruns	Costs exceed estimate	Regular spend review, control scope changes
Poor usability	Students struggle to book	Test prototypes with students, iterate

H. Testing and Quality Assurance
1. Functional testing
2. Concurrent booking testing
3. Identity testing
4. Authorization testing
5. Compatibility testing
6. Usability testing
7. Performance testing
8. Regression testing

I. Implementation and Maintenance
Before release, verify production backend, database, identity integration, access permissions, and monitoring. Include a deployment checklist, backup/recovery plan, and rollback procedure.

After release: monitor errors, review support requests, fix defects, apply security updates, collect user feedback. Possible future features: push notifications, waiting lists, room recommendations, and usage analytics.

Final Evaluation of Version 4:
Version 4 was the most complete response. It covered platforms, features, booking-conflict handling, identity verification, SDLC structure, roles, risks, testing, deployment, maintenance, and a preliminary budget. Because the prompt required assumptions to be labelled and university-specific details to be flagged, the response distinguished general recommendations from matters that require confirmation with university IT. The schedule and budget still require stakeholder review before use.

Comparison of the Four Versions:
Version	Approach	Main Output	Limitation
1	General instruction	Basic development steps	Too general to implement
2	Requirements-focused	Requirements, roles, open questions	No schedule or budget
3	SDLC planning	7-phase plan with time, deliverables, risks, roles	Technical controls and budget missing
4	Detailed, constraint-based	Complete plan with concurrency, identity, testing, budget	Estimates and university details still need validation

═══

OVERALL REFLECTION

═══

1. What makes a prompt effective?
An effective prompt gives the AI clear context, a defined task, and an expected output. It specifies the audience, format, level of detail, and any constraints. It avoids vague language

and, where useful, includes a role for the AI to adopt. In this lab, "Explain the SDLC" produced a generic answer, while a prompt specifying a role, audience, and format produced a focused, structured explanation.

2. Which prompting technique was most useful?
Role prompting combined with format instructions was the most useful. Assigning the AI a role (senior software engineer, requirements analyst, project manager) shaped the tone and vocabulary of the response. Specifying the format turned loose paragraphs into structured answers that were easier to read and reuse. Prompt iteration was also essential, because even a good first prompt can always be improved.

3. Why is context important when working with AI?
AI has no knowledge of the specific project, audience, or constraints unless they're stated in the prompt. Without context, it falls back on general knowledge and produces generic answers. Providing context — the audience, the system, the constraints, the purpose — narrows the AI's choices and produces answers that fit the real situation. In Exercise B, mentioning "technical and security risks" is what led to a recommendation of the Spiral model.

4. Why should we review AI-generated responses instead of accepting them automatically?
AI answers are plausible, not guaranteed correct. They may invent features that were never requested, miss requirements from the brief, misclassify information, or present assumptions as facts. In Exercise C, the AI suggested Should-Have and Nice-to-Have features the bookstore never asked for. In Exercise D, the budget and schedule were illustrative estimates, not confirmed figures. A human must check accuracy, question assumptions, and validate the output against the original brief.

5. How can AI support software engineers across the SDLC?
Planning: generate preliminary project plans, schedules, and risk lists.
Requirements Analysis: turn a brief into structured functional and non-functional requirements, and flag ambiguities.
System Design: explore architecture options, database structures, interfaces, and API designs.
Development: explain concepts, draft example code, and suggest implementation approaches.
Testing: propose test cases, edge cases, and security checks.
Implementation: prepare deployment checklists, release notes, and user documentation.
Maintenance: summarise bug reports, investigate problems, and suggest improvements.
Across all phases, AI is best used to accelerate routine work — while the engineer remains responsible for decisions, quality, and outcomes.

OVERALL CONCLUSION

Prompt engineering is an important skill in software engineering because it helps users communicate their needs clearly to AI tools. This assignment examined zero-shot prompting, role prompting, software development models, requirements analysis, and prompt iteration.

It demonstrated that a well-structured prompt produces more relevant, organised, and practical output than a generic instruction.

The university study-room booking application showed how prompts can be refined from a basic idea into a detailed preliminary project plan. The final version addressed all seven SDLC phases, mobile platforms, functional and non-functional requirements, double-booking prevention, student identity verification, team responsibilities, risks, testing, implementation, maintenance, and budget estimation.

However, AI-generated information must be reviewed and validated. Schedules, budgets, university-specific requirements, security controls, and technical designs should be confirmed before implementation. Effective prompt engineering, combined with human expertise and critical evaluation, can help produce higher-quality software engineering work.

END OF ASSIGNMENT